

Machine Learning Assignment 2

Important Instructions

- Implement ALL algorithms from scratch using ONLY:
 - NumPy
 - Pandas
 - Matplotlib
- Usage of the following is STRICTLY prohibited:
 - scikit-learn classifiers
 - clustering APIs
 - decision tree APIs
 - Naive Bayes APIs
- You may use scikit-learn ONLY for:
 - train-test split
 - evaluation metrics
- Decision Tree implementation MUST use:
 - C4.5 algorithm
 - Information Gain
 - Split Information
 - Gain Ratio
- ID3-only implementations will receive deductions.
- Submission must include:
 - PDF report
 - source code (.py or .ipynb)
 - dataset_generator.py
- Any unexplained code may be questioned during viva.
- Hardcoded outputs will result in zero marks.

Dataset Construction Guidelines

Students are required to generate datasets programmatically.

Reproducibility Requirement

Use:

`seed = last 3 digits of your roll number`

to ensure reproducibility.

Minimum Dataset Requirements

To ensure meaningful experimentation, ALL datasets must satisfy the following constraints unless explicitly stated otherwise.

- Minimum number of samples:

$$n \geq 1000$$

- Minimum number of features:

$$d \geq 15$$

- Every dataset must contain:

- at least 5 informative features,
- at least 5 noisy/irrelevant features.

- At least ONE dataset must contain:

$$d \geq 50$$

features.

- At least ONE dataset must contain:

$$n \geq 5000$$

samples.

- Extremely small toy datasets are NOT allowed except for:

- debugging,
- visualization,
- conceptual demonstrations.

- Direct usage of Kaggle datasets or public datasets is NOT allowed unless explicitly mentioned.

Dataset Diversity Requirement

Students must generate at least THREE dataset categories:

1. low-noise dataset,
2. high-noise dataset,
3. high-dimensional dataset.

Each dataset must demonstrate different model behavior.

Example: k-Means Friendly Dataset

The following example generates well-separated Gaussian clusters.

```
import numpy as np

np.random.seed(123)

cluster1 = np.random.normal(
    loc=[2,2],
    scale=0.5,
    size=(1000,10)
)

cluster2 = np.random.normal(
    loc=[8,8],
    scale=0.5,
    size=(1000,10)
)

X = np.vstack((cluster1, cluster2))
```

This dataset satisfies:

- spherical cluster assumption,
- balanced cluster sizes,
- clear separation.

Example: Correlated Features for Naive Bayes

```
X1 = np.random.normal(0,1,2000)

X2 = X1 + np.random.normal(0,0.1,2000)

X3 = np.random.normal(0,1,2000)
```

Here:

- X_1 and X_2 are highly correlated,
- Naive Bayes independence assumption is violated.

Example: High-Dimensional Tree Dataset

```
informative = np.random.normal(
    0,1,(5000,10)
)

noise = np.random.normal(
    0,1,(5000,50)
)

X = np.hstack((informative , noise))
```

This dataset tests:

- feature selection,
- overfitting behavior,
- split quality,
- robustness to noisy features.

Mandatory Analysis

For EVERY dataset students construct, they must explain:

- which assumptions are satisfied,
- which assumptions are violated,
- why the algorithm succeeds or fails,
- expected bias-variance behavior,
- expected overfitting or underfitting behavior.

Question 1: k-Means Clustering [30 Marks]

You must implement k-Means clustering completely from scratch.

Part A: Core Implementation

Implement:

- random centroid initialization,
- Euclidean distance computation,
- cluster assignment,
- centroid update,
- stopping criterion.

DO NOT use loops wherever vectorization is possible.

Part B: Adversarial Dataset Construction

Construct TWO datasets:

- (a) a dataset where k-Means performs extremely well,
- (b) a dataset where k-Means fails despite visually separable clusters.

Explain:

- why the failure occurs,
- which assumption of k-Means is violated,
- whether feature scaling changes the outcome.

Part C: Initialization Sensitivity

Run your implementation 20 times using different initializations.

- (a) plot convergence behavior,
- (b) compare final clustering costs,
- (c) explain why different initializations lead to different local minima.

Question 2: Gaussian Naive Bayes [30 Marks]

Part A: Implementation

Implement Gaussian Naive Bayes from scratch.

Your implementation must include:

- prior estimation,
- mean estimation,
- variance estimation,
- log-probability computation,
- numerical stability handling.

Part B: Correlated Features Experiment

Generate a synthetic dataset where:

- two features are highly correlated,
- one feature is redundant.

Train your model and analyze:

- confidence scores,
- prediction errors,
- miscalibration behavior.

Part C: Counterexample Challenge

Construct:

- (a) a dataset where Naive Bayes performs surprisingly well despite violated assumptions,
- (b) a dataset where Naive Bayes completely fails.

You must justify:

- why the model succeeds/fails,
- which assumptions are violated,
- why decision boundaries become problematic.

Part D: Conceptual Analysis

Answer the following:

- (a) Why can Naive Bayes outperform more sophisticated models on small datasets?
- (b) Explain mathematically why correlated evidence leads to overconfident predictions.

Question 3: Decision Trees using C4.5 [40 Marks]

Part A: C4.5 Implementation

Implement a binary decision tree classifier from scratch using:

- entropy,
- information gain,
- split information,
- gain ratio.

Your implementation must support:

- recursive splitting,
- stopping criteria,
- maximum depth,
- minimum samples per split.

Part B: Gain Ratio Analysis

Using your generated datasets:

- (a) compare Information Gain and Gain Ratio,
- (b) identify cases where Information Gain prefers poor splits,
- (c) explain why Gain Ratio improves split selection.

Part C: Overfitting Investigation

Train your tree using:

- very small depth,
- medium depth,
- very large depth.

Plot:

- training accuracy,
- validation accuracy.

Explain:

- bias-variance tradeoff,
- why deep trees memorize data,
- why shallow trees underfit.

Part D: Greedy Splitting Counterexample

Construct a dataset where greedy splitting produces a suboptimal tree.

You must:

- draw the resulting tree,
- explain why the greedy decision fails,
- explain what globally optimal splitting would require.

Part E: Noise Sensitivity

Introduce:

- label noise,
- outliers.

Analyze how the tree changes.

Explain:

- why trees are unstable learners,
- why small data changes drastically alter structure.

Submission Requirements

- Submit:
 - PDF report,
 - .py or .ipynb file,
 - dataset_generator.py.
- Report must include:
 - experimental setup,
 - graphs,
 - observations,
 - failure analysis,
 - implementation discussion.
- Missing experimental analysis will result in heavy deduction even if implementation is correct.